

Resumen de cambios en OpenMarkov

Período: 28 de marzo – 7 de abril de 2026

Manuel Arias — CISIAD, UNED

7 de abril de 2026

Índice

1. Visión general	3
2. Refactorización de Variable	3
2.1. Problema	3
2.2. Solución	5
2.3. Por qué aporta	6
3. Refactorización de Node	6
3.1. Problema	6
3.2. Solución	7
3.3. Por qué aporta	8
4. Análisis de ProbNet y extracción de responsabilidades	9
4.1. Problema	9
4.2. Solución (implementada parcialmente)	11
4.3. Por qué aporta	12
5. Refactorización de DiscretePotentialOperations	13
5.1. Problema	13
5.2. Solución	13
5.3. Por qué aporta	15
6. Rediseño de la jerarquía de potenciales	15
6.1. Problema	15
6.2. Solución	16
6.3. Por qué aporta	18
7. Refactorización del sistema de excepciones	18
7.1. Problema	18
7.2. Solución	19
7.3. Por qué aporta	20
8. Heurísticas de eliminación de variables	20
8.1. Por qué aporta	21
9. Corrección de bugs	21
10. Calidad de código transversal	22
11. Tests nuevos	22

12. Resumen de métricas	23
13. Trabajo pendiente	23

1. Visión general

En los últimos 10 días se han realizado **152 commits** distribuidos en **19 módulos**. El trabajo se ha organizado en seis ejes principales:

- Eje 1: Refactorización de clases core** — Extracción de responsabilidades de las clases dios (`Variable`, `Node`, `ProbNet`, `DiscretePotentialOperations`) y rediseño de la jerarquía de potenciales.
- Eje 2: Corrección de bugs** — 11 issues resueltos (#456, #472, #486, #496, #497, #504, #506, #520, #532, #562, separación de conjuntos en PC).
- Eje 3: Calidad de código** — 470 variables marcadas `final`, 178 imports no usados eliminados, 502 tags Javadoc añadidos, métodos redundantes eliminados.
- Eje 4: Tests** — Nuevos tests de invariantes algebraicos, tests de regresión de `tableProject`, tests de `GTablePotential`, tests unitarios de `Criterion`, `Finding` y aritmética de `TablePotential`.
- Eje 5: Rediseño de potenciales** — Reparentado de `GTablePotential`, interfaces de capacidad, encapsulación de `TablePotential.values`.
- Eje 6: Documentación técnica** — 12 documentos LaTeX con 51 diagramas UML PlantUML describiendo cada refactorización.

Distribución de commits por módulo:

Módulo	Commits
org.openmarkov.core	44
org.openmarkov.inference	19
org.openmarkov.gui	17
org.openmarkov.io	11
org.openmarkov.learning.algorithm	9
org.openmarkov.integrationtests	9
org.openmarkov (POM/docs)	8
org.openmarkov.learning.core	7
org.openmarkov.learning.gui	5
Otros 10 módulos (1–4 c/u)	23
Total	152

2. Refactorización de Variable

Documento completo: docs/refactorizacion-variable/refactorizacion-variable.tex

2.1. Problema

La clase `Variable` (986 líneas) mezclaba dos responsabilidades fundamentalmente distintas: ser un **objeto de dominio** (nombre, tipo, estados) y contener **lógica de edición del grafo** que dependía de `Node`, `ProbNet` y `Link`. Esto causaba:

- **Dependencia circular conceptual:** `Variable` (dominio) dependía de `Node` (grafo), pero `Node` contenía una `Variable`. Modificar la lógica de edición de estados requería entender y tocar la clase de dominio.

- **Sincronización frágil:** Los campos `name`, `baseName` y `timeSlice` se sincronizaban en 3 puntos distintos con side-effects: llamar a `setBaseName()` después de `setName()` podía dejar el estado interno inconsistente.
- **Ordenación no determinista:** `compareTo()` usaba `hashCode()`, lo que producía un orden no reproducible entre ejecuciones de la JVM.

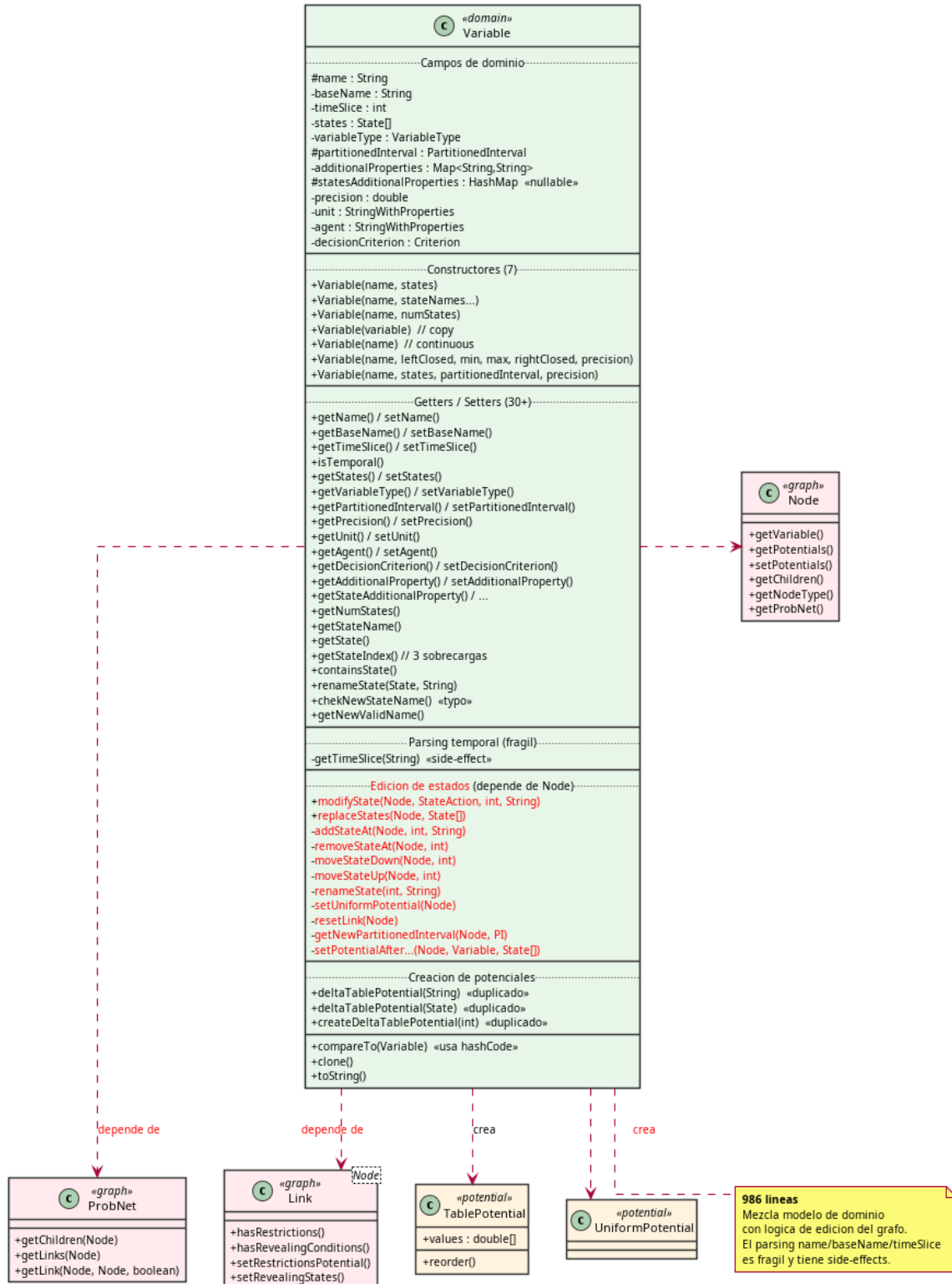


Figura 1: Variable antes de la refactorización: 986 líneas con 12 métodos de edición de estados (en rojo) que dependen de Node, ProbNet y Link. Nótese la anotación «typo» en `chekNewStateName()` y «usa hashCode» en `compareTo()`.

2.2. Solución

Extracción de los 12 métodos de edición de estados a una clase estática de utilidad `VariableStateOperations`, y consolidación de la sincronización temporal en dos métodos idempotentes.

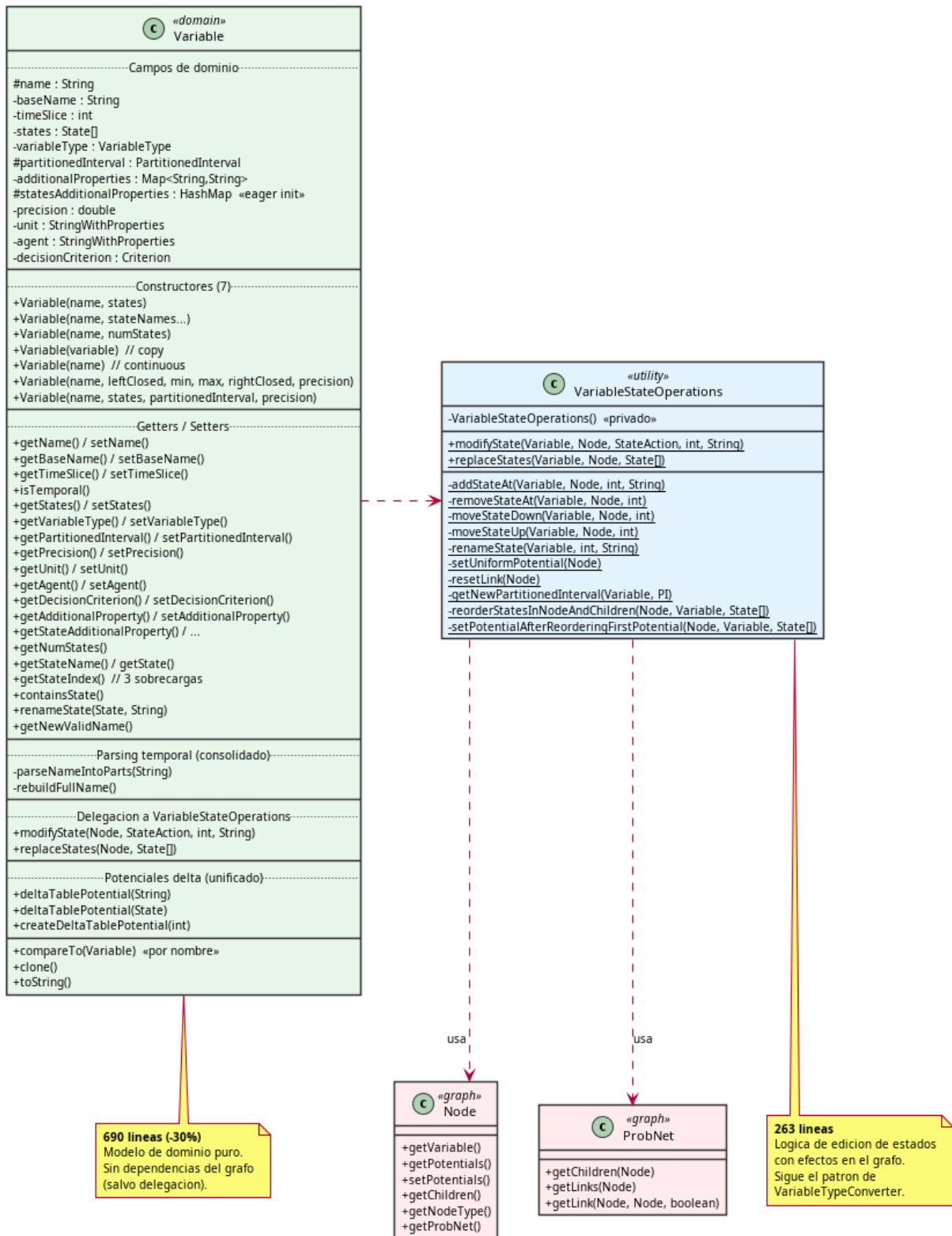


Figura 2: Variable después: 690 líneas (-30%). La lógica de edición de estados vive en `VariableStateOperations` (263 líneas). `Variable` queda como modelo de dominio puro con delegación. El parsing temporal se consolida en `parseNameIntoParts()` / `rebuildFullName()`.

2.3. Por qué aporta

1. **Testabilidad:** Los métodos de edición de estados ahora son estáticos y se pueden testear aisladamente sin construir un **Variable** completo. Los tests pasaron de 28 a 99.
2. **Mantenibilidad:** Modificar la lógica de edición ya no requiere tocar la clase de dominio. Un cambio en cómo se recalculan los potenciales tras añadir un estado solo afecta a **VariableStateOperations**.
3. **Determinismo:** El `compareTo()` por nombre alfabético es estable y reproducible, eliminando errores intermitentes en estructuras ordenadas (**TreeSet**, **SortedMap**).
4. **Robustez temporal:** La consolidación de `name`/`baseName`/`timeSlice` elimina la clase de bugs donde “X [3]” se parseaba como `baseName=“X”` pero luego `setBaseName(“Y”)` no actualizaba `name`.

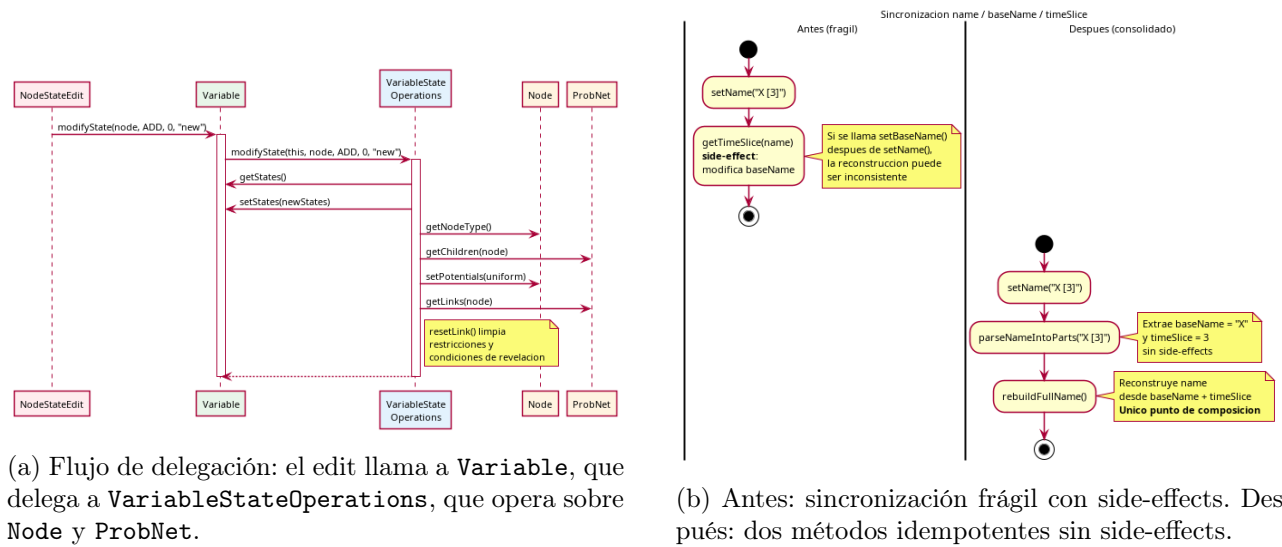


Figura 3: Diagramas de secuencia y actividad de la refactorización de **Variable**.

3. Refactorización de Node

Documento completo: docs/refactorizacion-node/refactorizacion-node.tex

3.1. Problema

La clase **Node** (1.037 líneas) tenía **6 razones para cambiar**: identidad del nodo, CRUD de potenciales, delegación al grafo, metadatos, presentación visual, y 308 líneas de lógica compleja de transformación (absorción, conversión de tipos, funciones de utilidad). Cada una de estas lógicas complejas era invocada por exactamente un caller externo, lo que hacía de **Node** un “vertedero” de funcionalidad.

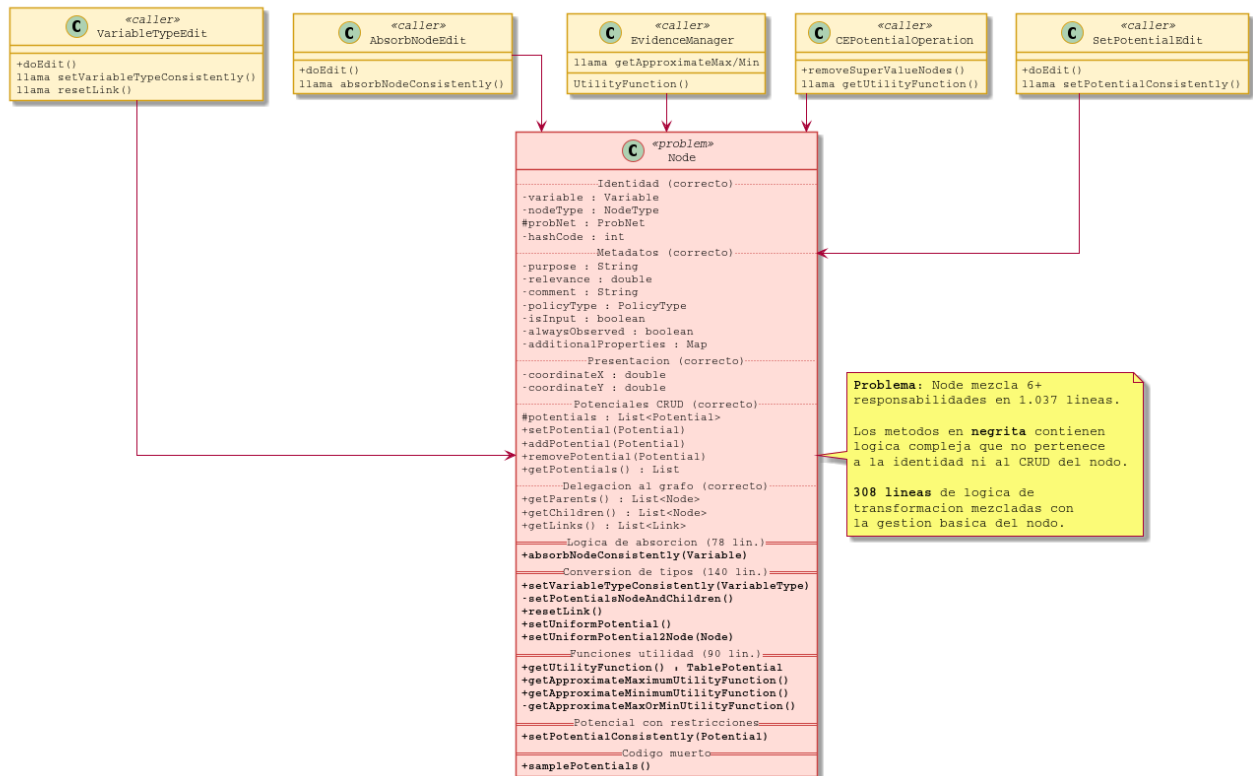


Figura 4: Node antes: 1.037 líneas con 5 bloques de lógica de transformación (en negrita) que no pertenecen a la identidad del nodo. Cada bloque es invocado por un único caller (amarillo).

3.2. Solución

Aplicación del patrón **Extract Class** en 5 fases, extrayendo cada bloque a la clase que lo necesita o a una clase de utilidad estática enfocada:

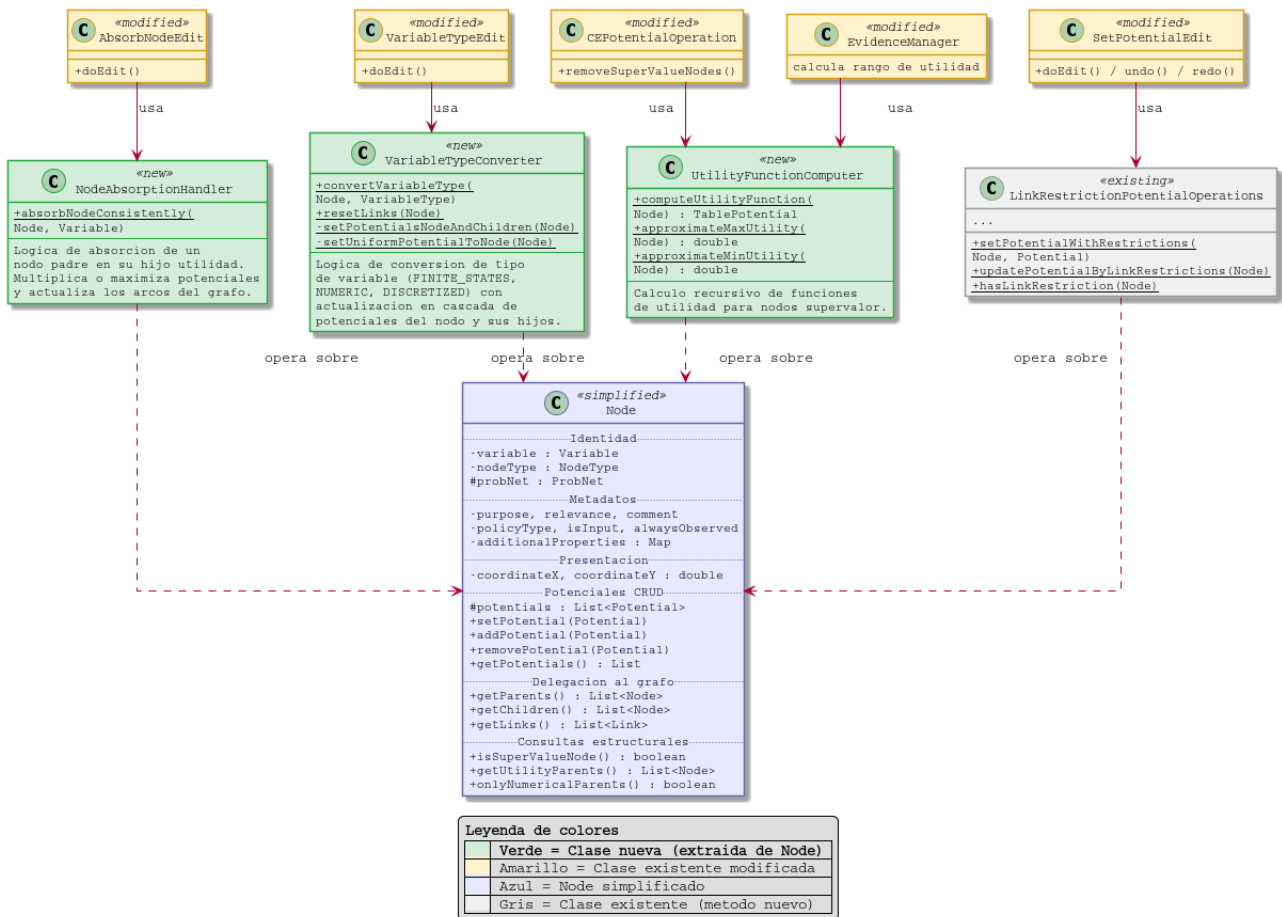


Figura 5: Node después: 670 líneas (–35 %). La lógica de transformación se ha extraído a 3 clases nuevas (verde) y 1 clase existente (gris). Cada caller ahora invoca directamente la clase especializada.

3.3. Por qué aporta

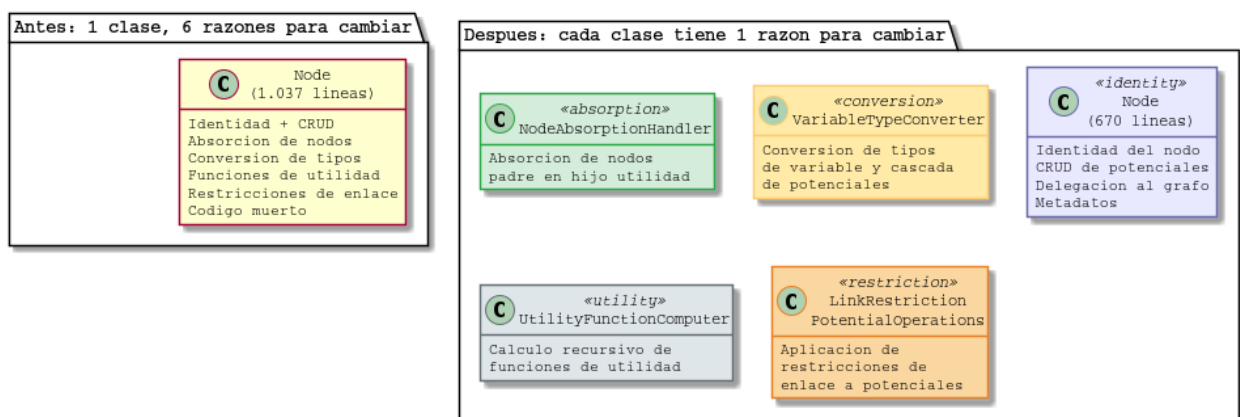


Figura 6: Aplicación del Principio de Responsabilidad Única: de 1 clase con 6 razones para cambiar a 5 clases con 1 razón cada una.

- SRP (Single Responsibility Principle):** Cada clase tiene ahora una única razón para cambiar. Un bug en la absorción de nodos solo requiere modificar NodeAbsorptionHandler, sin riesgo de afectar al CRUD de potenciales ni a la presentación visual.
- Cohesión:** absorbNodeConsistently() (78 líneas) operaba sobre potenciales, arcos y variables

— una operación completa de alto nivel que no tiene relación con la identidad de un nodo. Ahora vive donde le corresponde: en `NodeAbsorptionHandler`, junto a `AbsorbNodeEdit` que es su único usuario.

3. **Descubribilidad:** Antes, un desarrollador que buscara “absorción de nodos” lo encontraría enterrado a la mitad de una clase de 1.037 líneas. Ahora tiene una clase dedicada cuyo nombre anuncia su propósito.
4. **Sin rotura de API:** Los callers existentes (`AbsorbNodeEdit`, `VariableTypeEdit`, etc.) fueron modificados para usar las clases extraídas, pero la interfaz pública de `Node` se mantuvo intacta para el resto del sistema.

4. Análisis de ProbNet y extracción de responsabilidades

Documento completo: `docs/analysis-probnet/analysis-probnet.tex`

4.1. Problema

`ProbNet` es la clase más crítica de OpenMarkov: 1.440 líneas, 80+ métodos públicos, referenciada por 490 archivos. Es un ejemplo de libro de texto del antipatrón **God Class**: concentra nodos, links, variables, potenciales, constraints, agentes, clasificación de redes, y metadatos de I/O en una sola clase.

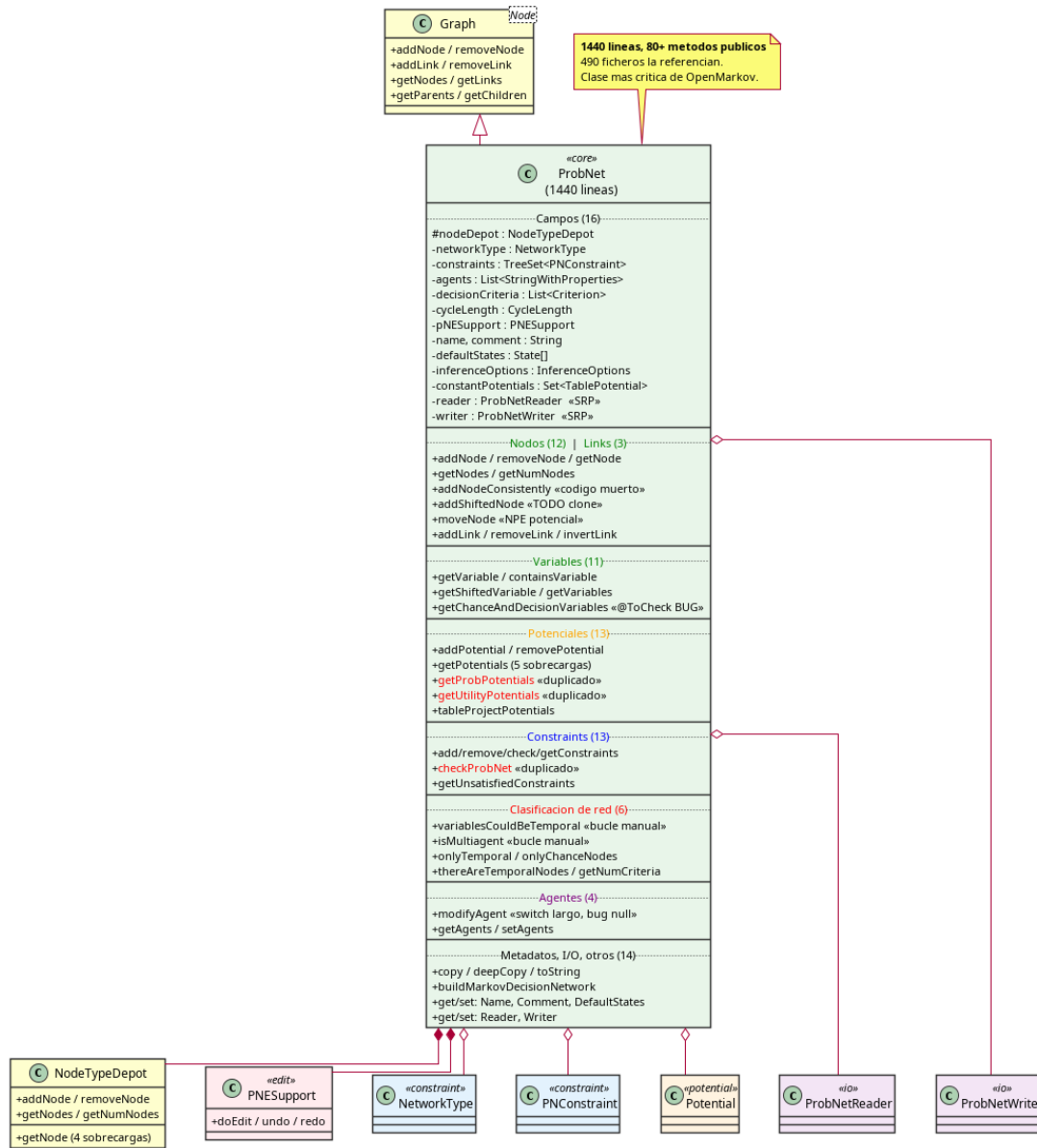


Figura 7: ProbNet actual: 1.440 líneas con 80+ métodos. Los colores indican responsabilidades: verde=nodos/links/variables, naranja=potenciales, azul=constraints, rojo=clasificación, púrpura=agentes. Los campos **reader/writer** (etiquetados SRP) acoplan el modelo de dominio al I/O.

El análisis identificó 4 bugs confirmados y 7 code smells:

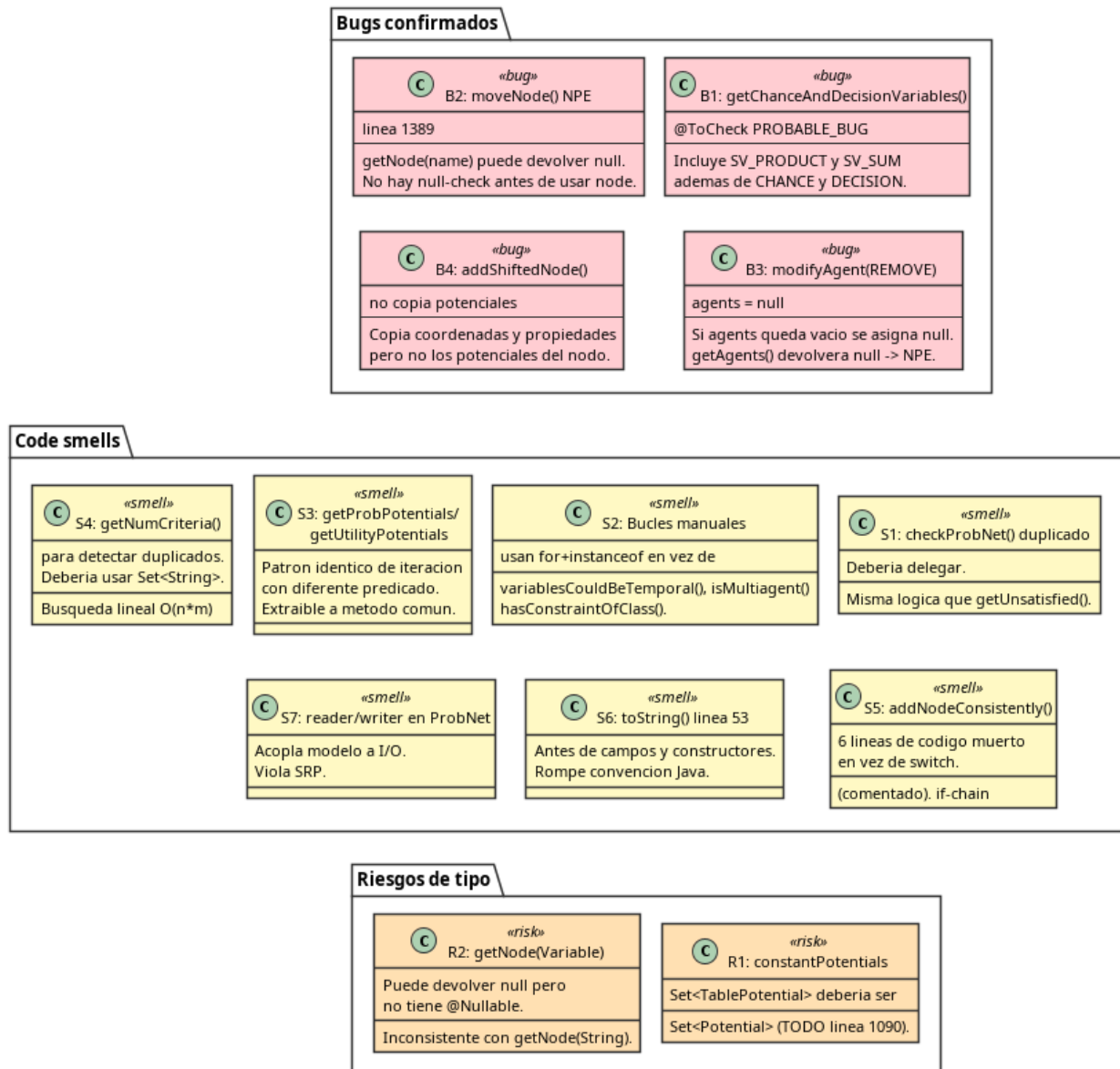


Figura 8: Bugs detectados en ProbNet (rojo), code smells (amarillo) y riesgos de tipo (naranja). B2 (moveNode() NPE) y B3 (modifyAgent(REMOVE) dejando null) son errores de null-safety; B4 (addShiftedNode()) pierde información; S7 (reader/writer) viola SRP y DIP.

4.2. Solución (implementada parcialmente)

Se extrajeron 3 clases de utilidad, reduciendo ProbNet en ~340 líneas:

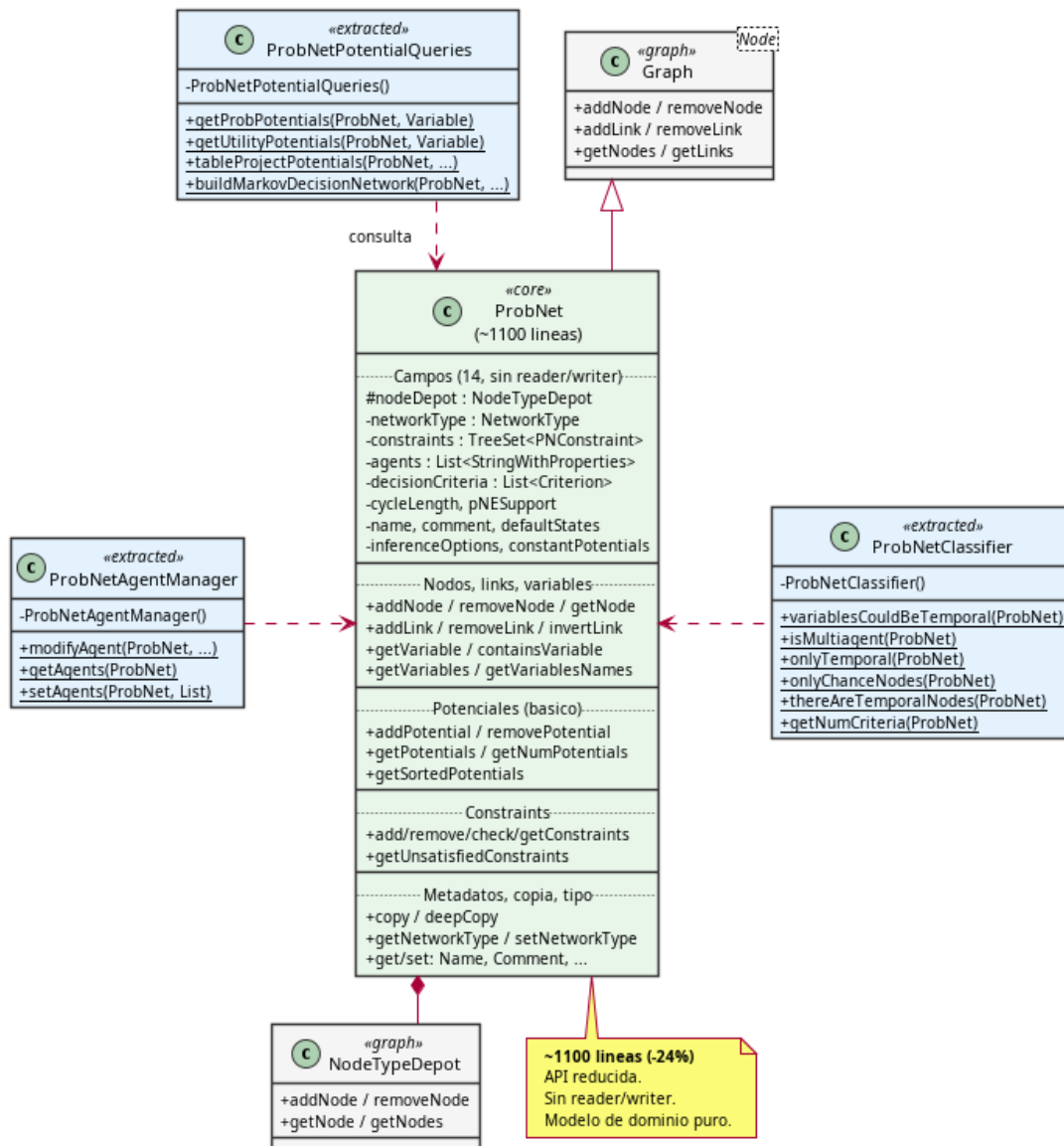


Figura 9: ProbNet propuesta (~1.100 líneas, -24 %): las responsabilidades de clasificación, gestión de agentes y consultas de potenciales se han extraído a clases estáticas dedicadas (azul). El modelo queda como grafo + constraints + metadatos.

4.3. Por qué aporta

1. **Reducción del radio de impacto:** Con 490 ficheros referenciando **ProbNet**, cualquier cambio tiene un potencial de propagación enorme. Extraer responsabilidades reduce el número de razones por las que hay que modificar la clase central.
2. **Testabilidad:** Los 6 métodos de clasificación (antes con 0 % de cobertura) ahora son estáticos en **ProbNetClassifier** y se pueden testear sin construir toda la infraestructura de **ProbNet**.
3. **Corrección de bugs:** La gestión de agentes (S3, con bug de null) fue extraída a **ProbNetAgentManager**, donde el bug fue visible y corregible sin riesgo de afectar nodos o potenciales.
4. **Desacoplamiento I/O:** La propuesta de eliminar **reader/writer** de **ProbNet** (fase 6, pendiente) eliminaría la violación del Dependency Inversion Principle: un modelo de dominio no debería depender de su mecanismo de persistencia.

5. Refactorización de DiscretePotentialOperations

Documento completo: docs/refactorizacion-discrete-potential-operations/

5.1. Problema

`DiscretePotentialOperations` (2.175 líneas) era la segunda clase más grande del core: 28 métodos públicos estáticos y ~15 privados, cubriendo aritmética, eliminación, maximización, fusión, transformación y factoría de potenciales. El problema no era solo el tamaño, sino que un bug en `normalize()` podía estar a 50 líneas de `multiplyAndMaximize()`, en un fichero donde era imposible navegar sin buscar.



Figura 10: Antes: una clase monolítica de 2.175 líneas con 6 responsabilidades mezcladas. Los métodos privados (~15) están compartidos sin criterio claro.

5.2. Solución

Aplicación del patrón **Facade**: `DiscretePotentialOperations` se mantiene como API pública (~460 líneas de delegación pura), pero la implementación se distribuye en 6 clases package-scoped organizadas por responsabilidad:

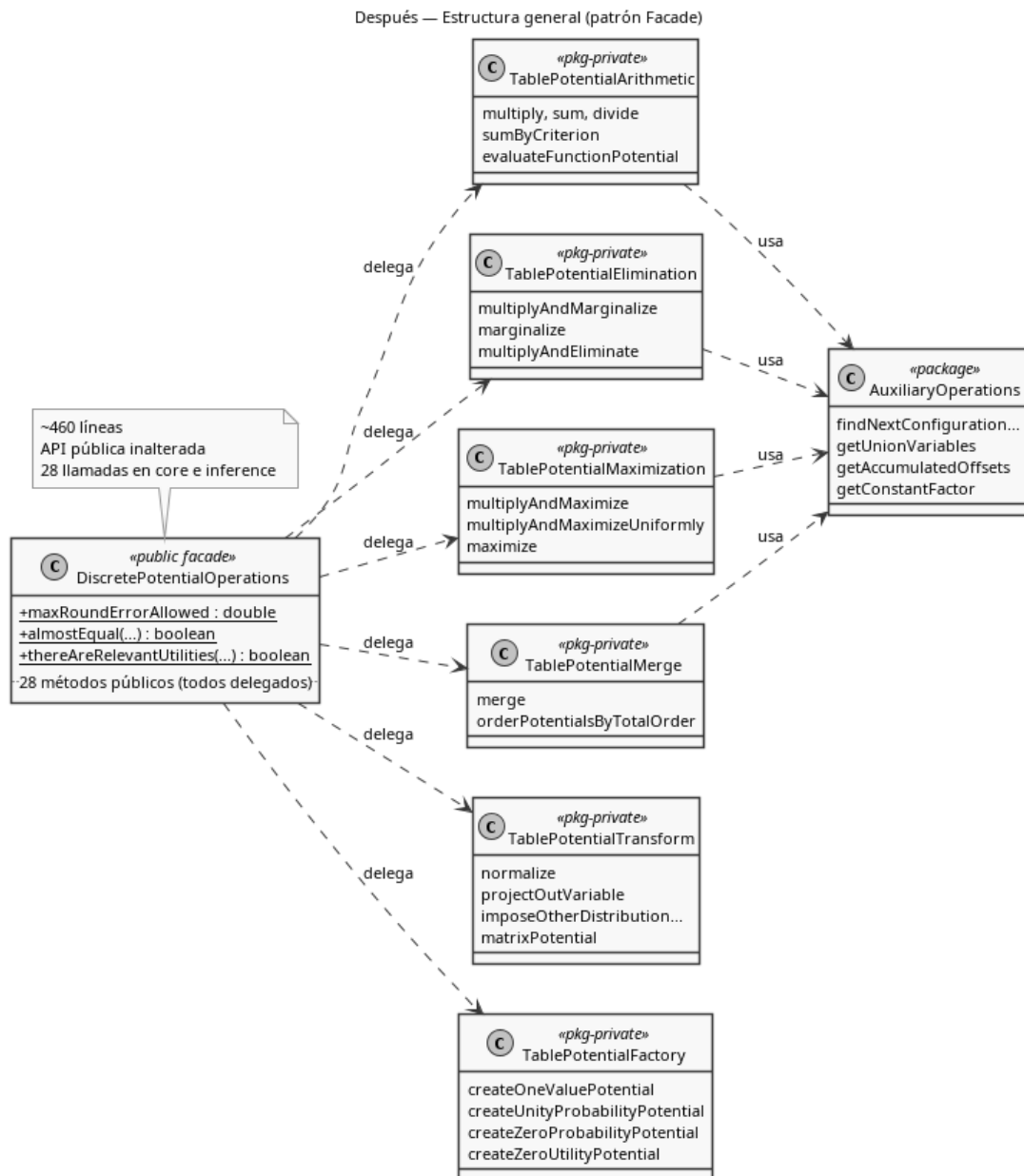


Figura 11: Después: Facade pública inalterada + 6 clases de implementación package-scoped. Cada clase contiene los métodos de una sola responsabilidad y sus helpers privados asociados.

Después — Métodos de las clases extraídas

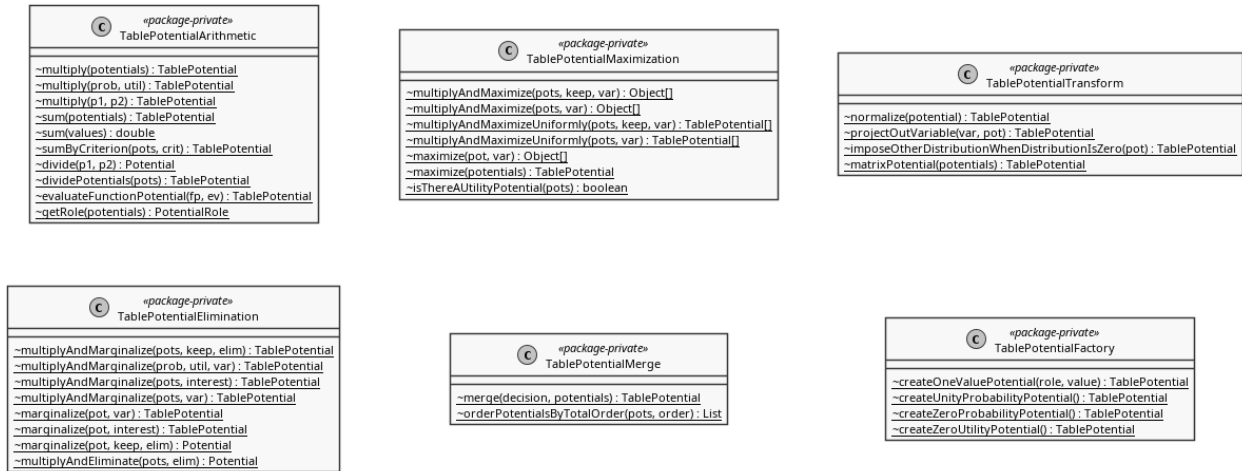


Figura 12: Detalle de los métodos de cada clase extraída: Arithmetic (10 métodos), Elimination (8), Maximization (7), Merge (2), Transform (4), Factory (4).

5.3. Por qué aporta

1. **Transparencia total:** Los 28 callers existentes en `core` e `inference` no requirieron ningún cambio. El Facade delega exactamente igual que antes.
2. **Localización de bugs:** Si hay un bug en la multiplicación de potenciales, solo hay que mirar `TablePotentialArithmetic` (~450 líneas), no 2.175 líneas. Los helpers privados están junto a los métodos que los usan.
3. **Navegabilidad:** Cada fichero tiene un propósito claro anunciado por su nombre. Un desarrollador nuevo puede entender la estructura en segundos.
4. **Paralelización futura:** Las clases concurrentes en `potential/operation/concurrent/` podrían extender las clases individuales en vez de duplicar la clase monolítica.

6. Rediseño de la jerarquía de potenciales

Documento completo: docs/rediseño-potenciales/rediseño-potenciales.tex

6.1. Problema

La jerarquía de potenciales tenía tres defectos estructurales:

1. **TablePotential sobrecargada:** Contenía `strategyTrees[]` y `uncertainValues[]`, ambos `null` en la inmensa mayoría de las instancias. Todos los métodos (`tableProject`, `reorder`, `deepCopy`) contenían **bloques condicionales triples**: “si tiene `strategies` haz A, si tiene `uncertainty` haz B, si no haz C”. Además, el constructor de copia hacía alias mutable compartido (`uncertainValues = pot.uncertainValues`): modificar la copia corrompía el original.
2. **GTablePotential heredaba de TablePotential:** Pero no usaba el array `values[]` — solo `elementTable`. Heredar de `TablePotential` reservaba `tableSize * 8` bytes de memoria que nunca se utilizaban.
3. **Sin abstracción intermedia:** Los campos de indexación (`dimensions[]`, `offsets[]`, `tableSize`) eran comunes a ambas familias pero estaban duplicados en `TablePotential`.

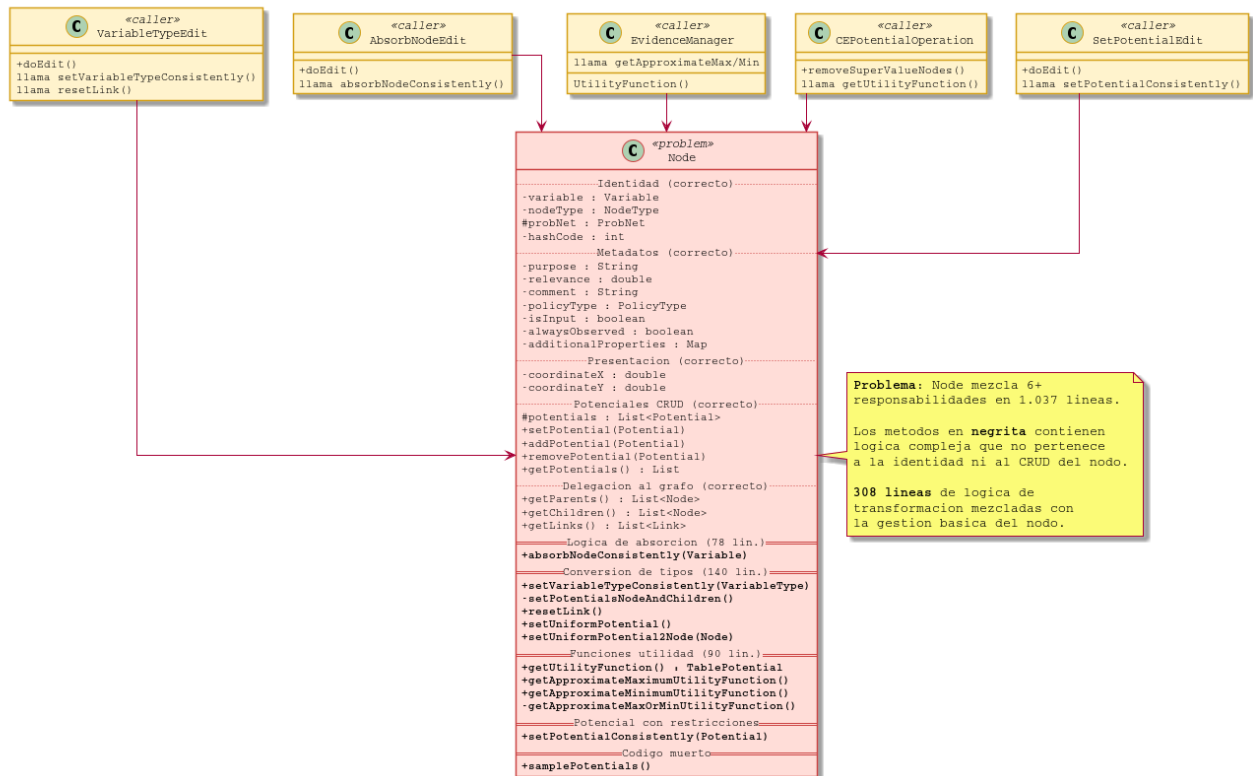


Figura 13: Antes: TablePotential contiene tres responsabilidades mezcladas (values, strategyTrees, uncertainValues). GTablePotential hereda un array values[] que nunca usa, con tableSize * 8 bytes de desperdicio por instancia.

6.2. Solución

Refactorización en 5 fases: extracción de AbstractIndexedPotential, split de TablePotential en 3 clases por el principio de Liskov, y reparentado de GTablePotential.

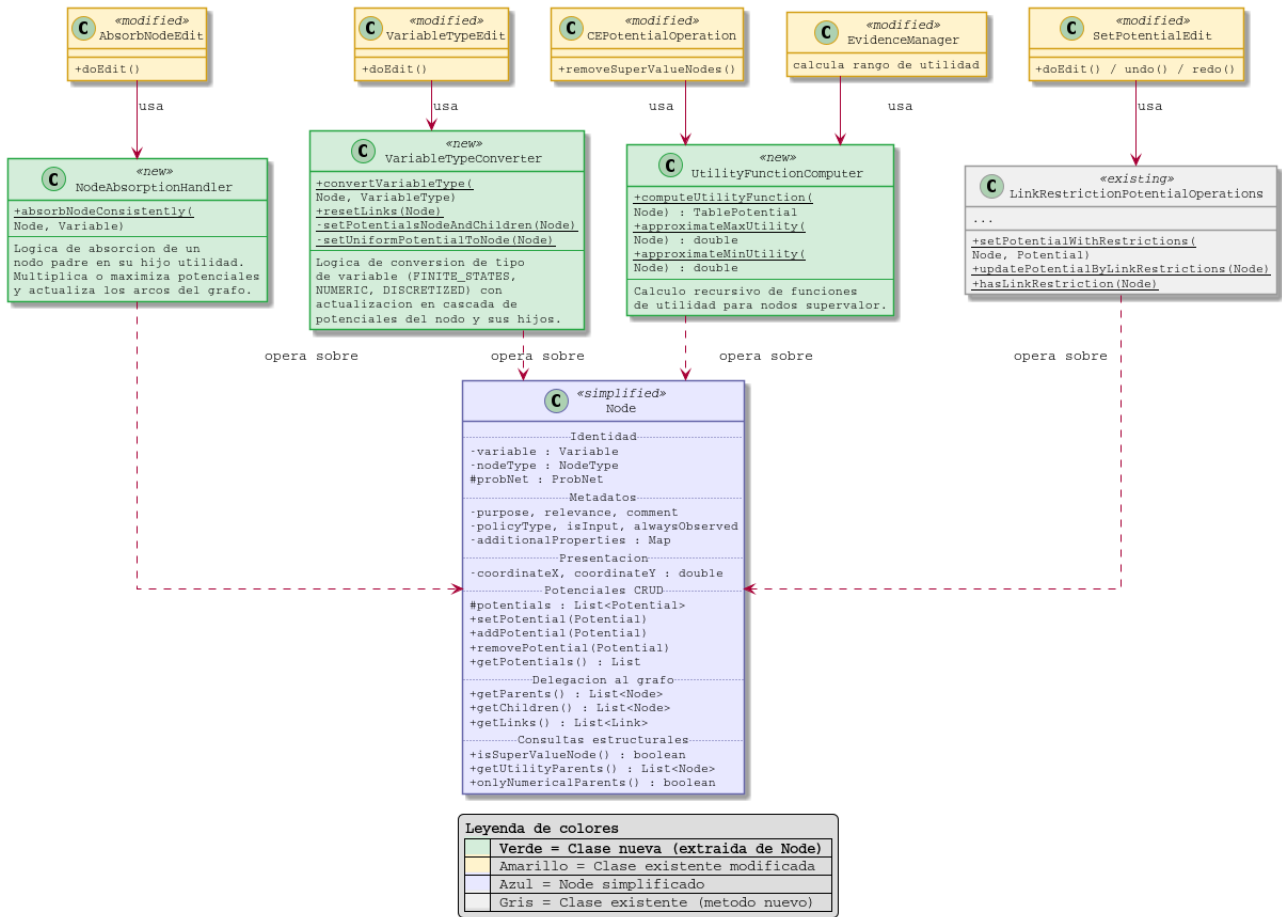


Figura 14: Después (fases 1–4): `AbstractIndexedPotential` captura la indexación compartida. `TablePotential` queda limpia (solo `values[]`). `StrategicTablePotential` y `UncertainTablePotential` encapsulan sus campos específicos. El despacho virtual elimina los bloques condicionales triples.

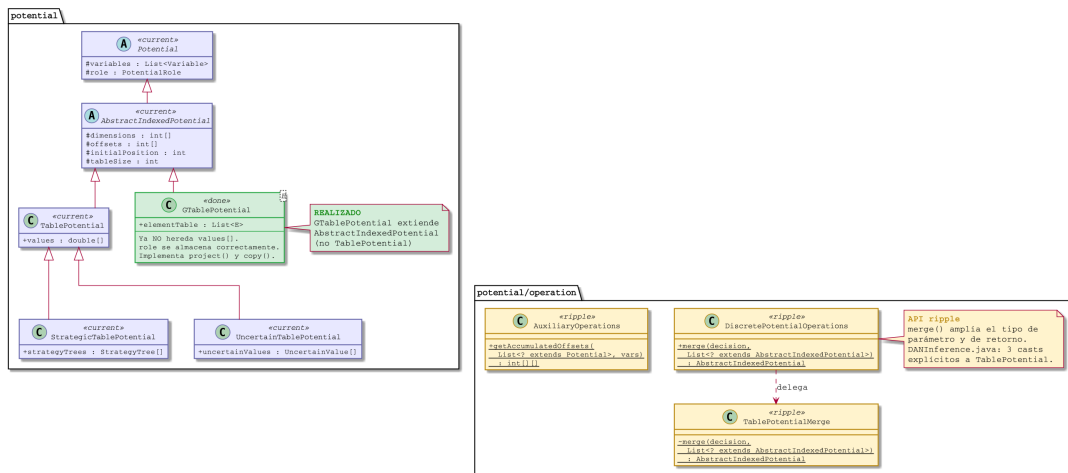


Figura 15: Fase 5 implementada: `GTablePotential` ahora extiende `AbstractIndexedPotential` directamente, sin heredar `values[]`. El *API ripple* en `merge()` fue contenido: solo 3 casts explícitos en `DANInference`.

6.3. Por qué aporta

1. **Principio de sustitución de Liskov:** Antes, `GTablePotential` violaba LSP: heredaba un contrato (`values[]` y sus métodos) que no cumplía. Ahora cada clase solo expone lo que realmente implementa.
2. **Eliminación de bloques condicionales:** Los triples `if (strategyTrees != null) ... else if (uncertainValues != null) ... else ...` en `tableProject()`, `reorder()`, y `deepCopy()` se reemplazan por despacho virtual: cada subclase implementa su versión, más simple y verificable.
3. **Bug de copia eliminado:** El alias mutable (`uncertainValues = pot.uncertainValues`) desaparece porque `UncertainTablePotential.deepCopy()` copia explícitamente su campo. Cada subclase es responsable de su propio `deepCopy`.
4. **Ahorro de memoria:** Cada instancia de `GTablePotential` (usada en CEA con muchos objetos `Choice`) ahorra `tableSize * 8` bytes de arrays `double[]` no utilizados.
5. **Extensibilidad:** Añadir un nuevo tipo de potencial indexado (por ejemplo, `SparseTablePotential`) solo requiere extender `AbstractIndexedPotential`, sin modificar `TablePotential`.

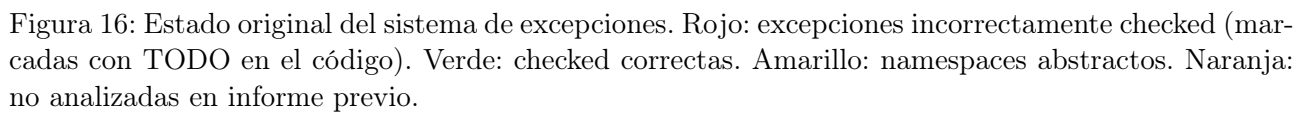
7. Refactorización del sistema de excepciones

Documentos: `docs/refactorizacion-excepciones-previo/` (análisis) y `docs/refactorizacion-excepciones-p` (resultado)

7.1. Problema

El sistema de excepciones tenía 5 problemas identificados:

1. 5 excepciones checked que deberían ser unchecked (el caller no puede recuperarse) contaminaban 53 ficheros con `throws` clauses innecesarias.
2. `SAXException` fugaba desde el parser XML hasta la GUI (`MainPanelListenerAssistant`), exponiendo detalles de implementación.
3. `NonProjectablePotentialException` era una clase plana con 8 causas distintas sin diferenciar — imposible hacer `catch` específico.
4. 34 bloques `try/catch` repetitivos en `actionPerformed()`.
5. Excepciones inalcanzables: `ThereIsNoNextEvidenceCaseException` nunca se lanzaba porque los botones se deshabilitaban preventivamente.



(a) Jerarquía checked final: OpenMarkovException como base, UserInputException para errores del usuario, DoEditException para fallos en edits.

(b) Jerarquía unchecked: OpenMarkovRuntimeException con 4 familias sealed. UnreachableException fuera de la jerarquía de dominio.

19

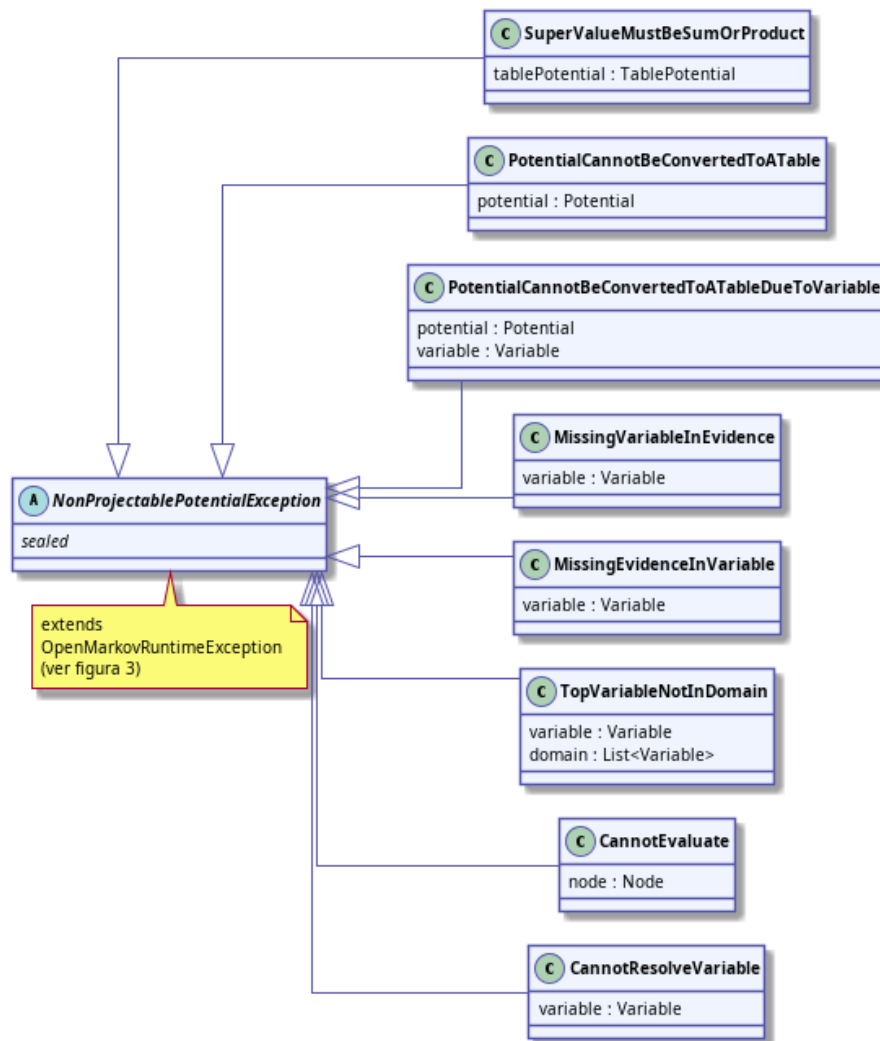


Figura 18: `NonProjectablePotentialException` como clase `sealed` con 8 subtipos semánticos. Cada subtipo lleva la información contextual necesaria (el potencial, la variable, el nodo) para un diagnóstico preciso.

7.3. Por qué aporta

1. **Limpieza de throws:** 40 cláusulas `throws` eliminadas en 53 ficheros. Cada método de inferencia ya no obliga a sus callers a manejar excepciones de las que no pueden recuperarse.
2. **Encapsulación:** La `SAXException` ya no llega a la GUI; el parser la envuelve en `ParserException`. Si mañana se cambia JDOM2 por StAX, la GUI no se ve afectada.
3. **Diagnóstico preciso:** Con la `sealed` hierarchy, un `catch` (`NonProjectablePotentialException e`) puede hacer `switch` para distinguir las 8 causas. Antes, el mensaje de error era siempre genérico.
4. **Sealed classes (Java 17):** El compilador garantiza que el `switch` sobre los subtipos es exhaustivo. Si se añade un 9.^o subtipo, todos los switches se marcan como incompletos en compilación.

8. Heurísticas de eliminación de variables

Documento completo: docs/heuristic/heuristics.tex

Se corrigieron 2 bugs en `CanoMoralElimination` y se añadieron 4 nuevas heurísticas (1 greedy + 3 lookahead):

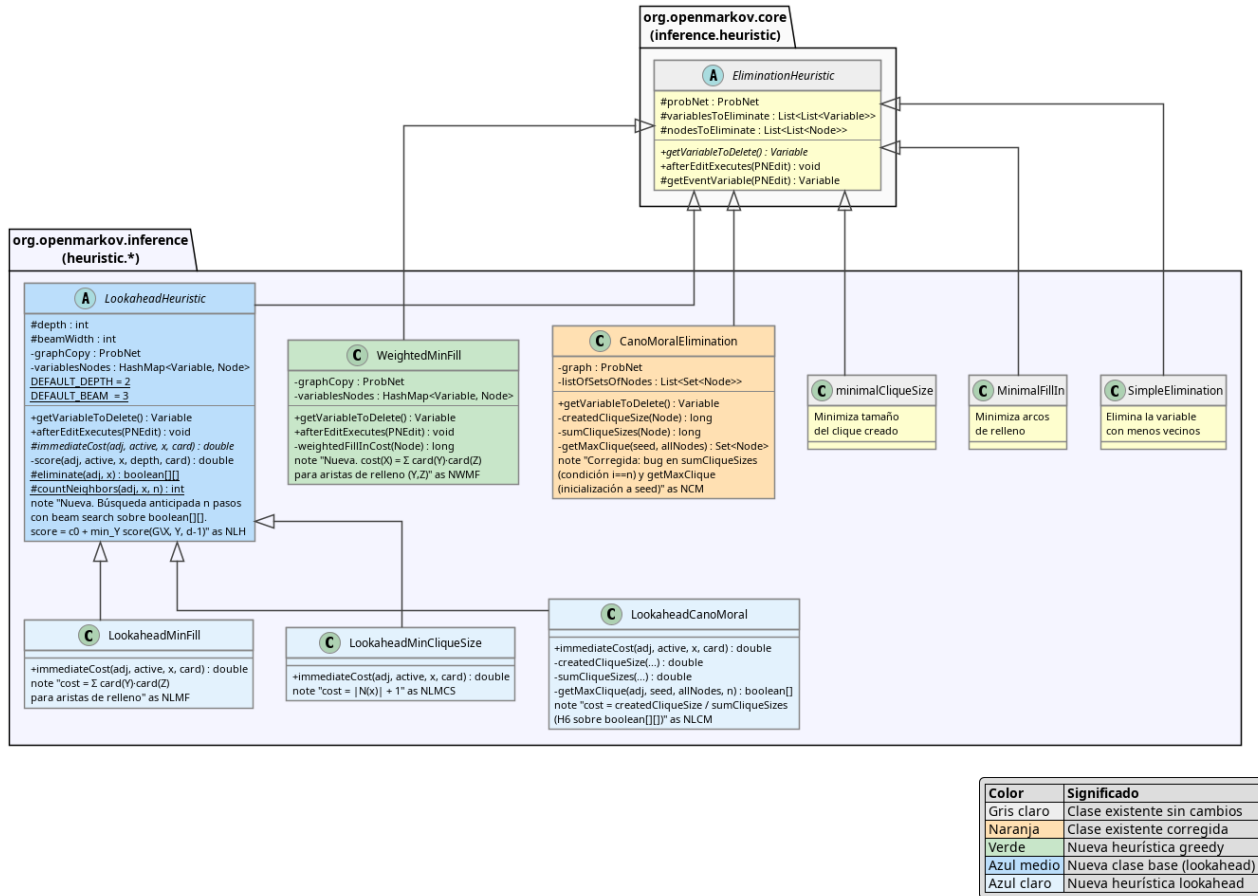


Figura 19: Heurísticas de eliminación. Naranja: **CanoMoralElimination** corregida (bug en `sumCliqueSizes` y en `getMaxClique`). Verde: **WeightedMinFill** (nueva greedy). Azul: framework **LookaheadHeuristic** con 3 implementaciones que miran n pasos adelante con beam search.

8.1. Por qué aporta

1. **Bug en CanoMoral:** `sumCliqueSizes()` siempre devolvía 0 (condición `i < numMaximalCliques` en vez de `i == numMaximalCliques`), haciendo que la heurística usara solo `createdCliqueSize` sin su divisor. El segundo bug (`getMaxClique` inicializado a `null`) causaba NPE cuando el clique semilla era el máximo.
2. **WeightedMinFill:** Supera a **MinimalFillIn** en redes donde los nodos tienen cardinalidades muy dispares (frecuente en redes médicas), porque pondera el relleno por el producto de cardinalidades en vez de contar solo aristas.
3. **Lookahead:** En la red **BN-hepar** (70 nodos), **LookaheadMinFill** reduce el coste del árbol de unión de 713 a 628 (-12%) respecto a **MinimalFillIn**, a cambio de un tiempo de compilación aceptable gracias al beam search.

9. Corrección de bugs

Issue	Descripción	Solución
#456	Error al abrir propiedades de nodo con potencial Weibull Hazard	Fix en GUI del diálogo de propiedades de nodo

Issue	Descripción	Solución
#472	<code>InvertLinkAndUpdatePotential</code> Se editó <code>redo()</code> asignaba potencial hijo al nodo incorrecto	Además, el <code>PotentialRole</code> heredado podía ser <code>JOINT_PROBABILITY</code> en vez de <code>CONDITIONAL_PROBABILITY</code>
#486	Error al evaluar DAN con 2 nodos de utilidad	Fix en inferencia DAN y test de integración
#496	Valores de coste/efectividad ausentes en árboles de decisión CEA	Fix en generación de árboles de decisión CE
#497	Árbol de estrategia mostraba <code>toString()</code> verbose	Fix en <code>toString()</code> de nodos de estrategia
#504/#532	Redondeo inconsistente en resultados de CEA	Fix en <code>inference + costEffectiveness</code>
#506	Absorción de nodo decisión permitida con padres desconocidos para el decisor	Verificación de predecesor informacional en <code>AbsorbNodeValidator</code>
#520	Error al evaluar red con nodo utilidad <code>SumPotential</code>	Fix en evaluación y test de integración
#562	<code>reorder()</code> devolvía <code>null</code> para <code>AugmentedProbTable</code>	Implementación de <code>reorder()</code> en 12 subclases de <code>Potential</code>
—	PC Algorithm: <code>evaluateSeparationSets</code> filtraba durante descubrimiento	Filtro movido a visualización
—	<code>multiply</code> devolvía <code>null</code> para lista vacía	Fix del caso borde

10. Calidad de código transversal

Cambios aplicados simultáneamente a todos los módulos (propagados el 2–3 de abril):

- **470 variables marcadas final** (13 módulos).
- **178 imports no usados eliminados** (12 módulos).
- **502 tags Javadoc añadidos** (`@param`, `@return`, `@throws`) en 14 módulos.
- **Autoría actualizada:** Tag `@author` `marias` → `Manuel Arias`.
- **5 PNConstraint subclasses adaptadas** a la nueva API tras extracción de `GraphNetwork` — propagado a todos los módulos.
- **Log level corregido:** root logger cambiado de `TRACE` a `INFO`.
- **PGMXReader_0_2:** Dispatch basado en reflexión reemplazado por interfaz funcional tipada.
- **hasValidStructure/isValidNode:** Corregidos — antes devolvían siempre `true`.

11. Tests nuevos

- **core — Invariantes algebraicos:** `DiscretePotentialOperationsAlgebraicInvariantsTest` — marginales suman 1, normalize preserva estructura, multiply asociativa.
- **core — Regresión tableProject:** Todos los patrones de proyección cubiertos.

- **core** — **Tests unitarios:** `CriterionTest`, `FindingTest`, `TablePotentialArithmeticEdgeCasesTest`.
- **core** — **Pre-reparenting:** Coverage de `GTablePotential`, `Choice` y flujos CEA antes del reparentado.
- **integrationtests** — **Invariantes de probabilidad:** Verifica invariantes para **todas** las variables de cada BN.
- **integrationtests** — **PGMXReadersTest:** Adaptado para 8 tipos de potencial sin parsers directos.
- **integrationtests** — **Issues:** Tests de regresión para #486 y #520.

12. Resumen de métricas

Métrica	Valor
Commits totales (10 días)	152
Módulos afectados	19 de 21
Bugs corregidos (issues)	11
Variables marcadas final	~470
Imports eliminados	178
Tags Javadoc añadidos	502
Clases extraídas (Extract Class)	12
Clases nuevas de test	~10
Tests nuevos	~100+
Diagramas UML (PlantUML)	51
Documentos técnicos (LaTeX)	12

13. Trabajo pendiente

1. **ProbNet** — **fases 5–6:** Evaluar extracción de `reader/writer` (alto riesgo, pendiente).
2. **Tests de ProbNet:** Llevar cobertura al objetivo >80%. Gaps críticos: clasificación, agentes, I/O, metadatos.
3. **Edits concretos:** 55 edits con solo 1 fichero de test. Cada edit debería tener tests propios.
4. **EM Algorithm:** Sigue deshabilitado. Pendiente de decisión sobre su futuro.
5. **Candidatos a diagramas UML (23):** Identificados en la auditoría de Javadoc pero aún sin implementar la mayoría.